Digital Signal Processing **HK261** **CO2036**

DIGITAL SIGNAL PROCESSING LAB REPORT

Lab 4 – Z-Transform and Discrete-Time Systems Analysis

Âu Lê Quang Trí - Student ID: 2453300

Nguyễn Lê Khải Tú - Student ID: 2453384

November 2, 2026

Overview

This report presents the results of Lab 4 of the Digital Signal Processing laboratory. The laboratory focuses on transform-domain and time-domain analysis of discrete-time signals and systems. The first part addresses the bilateral Z-transform and the determination of the Region of Convergence (ROC) for finite-duration, right-sided exponential, and two-sided sequences. The second part investigates multimedia processing capabilities in Scilab through image processing toolboxes, followed by detailed derivations and computational implementations of linear convolution and circular convolution using both the "folding and shifting" approach and the matrix formulation, alongside signal energy evaluations and stem-plot visualizations.

Table 1: Content of the report

Part	Content	Section
Exercise 1	Z-Transform and ROC of a finite-duration signal	4.1
Exercise 2	Z-Transform and ROC of right-sided exponential signals	4.2
Exercise 3	Z-Transform and ROC of a two-sided signal	4.3
Exercise 4	Investigation of Scilab audio processing toolbox	4.4
Exercise 5	Investigation of Scilab image processing toolbox	4.5
Exercise 6	Linear convolution (derivation, folding-shifting, matrix method)	4.6
Exercise 7	Circular convolution (derivation, folding-shifting, matrix method)	4.7

1 Introduction

In previous laboratory sessions, discrete-time signals and elementary operations were analyzed primarily in the time domain. Lab 4 extends this foundation into transform-domain analysis and comprehensive system convolution. The fundamental tool for studying discrete-time linear time-invariant (LTI) systems is the **Z-transform**, which plays an analogous role to the Laplace transform in continuous-time systems.

For a discrete-time sequence $x(n)$, the bilateral Z-transform $X(z)$ is defined as

$$X(z) = \mathcal{Z}\{x(n)\} = \sum_{n=-\infty}^{\infty} x(n)z^{-n}, \tag{1}$$

where $z = re^{j\omega}$ is a complex variable. Because an algebraic expression $X(z)$ does not uniquely specify a time-domain sequence without specifying the convergence domain of the infinite series, the transform must be paired with its **Region of Convergence (ROC)**:

$$\text{ROC} = \left\{ z \in \mathbb{C} : \sum_{n=-\infty}^{\infty} |x(n)z^{-n}| < \infty \right\}. \tag{2}$$

The geometry and boundary of the ROC determine fundamental system properties such as causality and bounded-input bounded-output (BIBO) stability.

In system analysis, computing the response of an LTI system to an arbitrary input $x(n)$ requires **linear convolution**:

$$y(n) = x(n) * h(n) = \sum_{k=-\infty}^{\infty} h(k)x(n-k), \tag{3}$$

where $h(n)$ is the unit impulse response. When dealing with periodic signals or block-based frequency-domain processing, **circular convolution** is employed:

$$y(n) = x(n) \circledast h(n) = \sum_{k=0}^{N-1} h(k)x((n-k) \bmod N). \tag{4}$$

In this laboratory, these analytical principles are demonstrated alongside practical implementations and toolbox investigations in Scilab.

2 Objectives

- Review theoretical foundations of the bilateral Z-transform and Region of Convergence (ROC).
- Determine the algebraic Z-transforms and sketch the corresponding ROCs for finite-duration, right-sided, and two-sided discrete-time sequences.
- Investigate existing Scilab libraries and toolboxes for digital image processing (such as IPCV or SIVP) and demonstrate representative image processing tasks.
- Analyze convergence conditions for linear convolution and derive system response formulations for causal and finite-impulse-response (FIR) cases.
- Implement linear convolution in Scilab using both the "folding and shifting" method and the matrix formulation.
- Implement circular convolution in Scilab using both the "folding and shifting" method and the matrix formulation.
- Calculate signal energies and graphically present discrete-time input, impulse response, output signals, and energy metrics within unified figures.

3 Methodology

Scilab serves as the computational environment for simulating discrete-time operations and verifying analytical derivations. In this laboratory, Scilab is utilized for:

- **Convolution Algorithms**: Implementing both linear convolution and circular convolution using vector-level shifting and element-wise products (the "folding and shifting" approach) as well as linear algebra formulations (constructing Toeplitz convolution matrices and circulant matrices).
- **Signal Energy Computation**: Evaluating the total energy of a discrete-time signal $E_x = \sum_{n=-\infty}^{\infty} |x(n)|^2$ using Scilab's vectorized arithmetic: `Ex = sum(abs(xn).^2)`.
- **Image Processing Exploration**: Leveraging dedicated image processing toolboxes in Scilab to read, manipulate (e.g., spatial filtering, histogram equalization), and visualize digital images.

3.1 Common conventions

All computational scripts and graphical outputs adhere to the following conventions:

- A discrete-time signal is represented as a sample vector `xn` accompanied by an origin index `xorigin`, indicating the 1-based vector position corresponding to $n = 0$. The time index vector is defined as

$$n_i = i - \texttt{xorigin}, \qquad i = 1, 2, \ldots, \texttt{length(xn)}. \tag{5}$$

- The origin position of a signal is mathematically highlighted with an arrow underneath the sample at $n = 0$, denoted as $x(\underset{\uparrow}{0})$.
- For linear convolution between sequences $x(n)$ of length N_x with origin at index `xorigin` and $h(n)$ of length N_h with origin at index `horigin`, the resulting sequence $y(n)$ has length $L = N_x + N_h - 1$, and its origin index is located at:

$$\texttt{yorigin} = \texttt{xorigin} + \texttt{horigin} - 1. \tag{6}$$

- For circular convolution of length N, indices are wrapped cyclically using modulo arithmetic: $(n - k) \bmod N$.
- Signals are visualized using stem plots via Scilab's `plot2d3` combined with marker symbols in shared `subplot` panels, ensuring consistent axes for direct visual comparison of inputs, impulse responses, outputs, and energy values.

4 Practical Contents

4.1 Exercise 1

Question: For the discrete-time signal $x(n)$, find the corresponding Z-Transform and its Region of Convergence (ROC):

$$x(n) = 2\delta(n+2) - \delta(n+1) + 2\delta(n) - 3\delta(n-1) + 4\delta(n-2)$$

Answer: The given sequence $x(n)$ is a finite-duration, two-sided sequence composed of shifted and scaled unit impulse functions $\delta(n - n_0)$. The sample values across the integer support $n \in [-2, 2]$ are explicitly given by:

$$\begin{aligned} n = -2 &: & x(-2) &= 2, \\ n = -1 &: & x(-1) &= -1, \\ n = 0 &: & x(0) &= 2, \\ n = 1 &: & x(1) &= -3, \\ n = 2 &: & x(2) &= 4, \end{aligned}$$

and $x(n) = 0$ for all $|n| > 2$. In vector notation with the origin marker ($\uparrow$) positioned at $n = 0$:

$$x(n) = [2, -1, \underset{\uparrow}{2}, -3, 4].$$

4.1.0.1 Z-Transform Derivation:

By definition of the bilateral Z-transform:

$$X(z) = \mathcal{Z}\{x(n)\} = \sum_{n=-\infty}^{+\infty} x(n) z^{-n}.$$

Using the sifting property of the unit impulse function $\mathcal{Z}\{\delta(n-n_0)\} = z^{-n_0}$ and applying linearity:

$$\begin{aligned} X(z) &= \sum_{n=-2}^{2} x(n)z^{-n} \\ &= x(-2)z^2 + x(-1)z^1 + x(0)z^0 + x(1)z^{-1} + x(2)z^{-2} \\ &= 2z^2 - z + 2 - 3z^{-1} + 4z^{-2}. \end{aligned}$$

Expressing $X(z)$ in rational form with a common denominator of z^2:

$$X(z) = \frac{2z^4 - z^3 + 2z^2 - 3z + 4}{z^2}. \tag{7}$$

4.1.0.2 Region of Convergence (ROC):

The Region of Convergence is defined as the set of complex numbers z for which the power series converges absolutely, i.e., $\sum_{n=-2}^{2} |x(n)z^{-n}| < \infty$:

- Due to the non-causal components ($n < 0$), namely the polynomial terms $2z^2 - z$, as $|z| \to \infty$, $|X(z)| \to \infty$. Consequently, the point at infinity $z = \infty$ does not belong to the ROC.
- Due to the causal components ($n > 0$), namely $-3z^{-1} + 4z^{-2} = -\frac{3}{z} + \frac{4}{z^2}$, as $|z| \to 0$, $|X(z)| \to \infty$ due to the double pole at the origin $z = 0$. Consequently, the point $z = 0$ does not belong to the ROC.
- For any finite, non-zero complex value z ($0 < |z| < \infty$), the sum comprises a finite number of finite terms and is therefore strictly bounded.

Thus, the Region of Convergence is:

$$\text{ROC} : 0 < |z| < \infty \quad \text{(the entire complex } z\text{-plane except } z = 0 \text{ and } z = \infty). \tag{8}$$

Remark on Stability and Fourier Transform: Because the ROC includes the unit circle $|z| = 1$, the Discrete-Time Fourier Transform (DTFT) of $x(n)$ exists and is obtained directly by evaluating $X(z)$ on the unit circle $z = e^{j\omega}$:

$$X(e^{j\omega}) = 2e^{j2\omega} - e^{j\omega} + 2 - 3e^{-j\omega} + 4e^{-j2\omega} = 2 - 4j\sin(\omega) + 4\cos(2\omega) + 2j\sin(2\omega).$$

4.2 Exercise 2

Question: Find the z-Transform and the ROC of the following discrete-time signal

$$x(n) = 0.5^n u(n) + 0.4^n u(n)$$

Answer: At first glance, the signal $x(n)$ exhibits linearity property. Let

$$x_1(n) = 0.5^n u(n) \quad \text{and} \quad x_2(n) = 0.4^n u(n)$$

where $u(n)$ is the unit step signal.

Let's take the z-Transform of $x_1(n)$, which are

$$X_1(z) = \sum_{n=-\infty}^{+\infty} [0.5^n u(n)]\, z^{-n} = \sum_{n=0}^{+\infty} \left(0.5z^{-1}\right)^n$$

This geometric series converges if and only if

$$|0.5z^{-1}| < 1 \Rightarrow |z| > 0.5$$

So $\text{ROC}_1 : |z| > 0.5$. For $x_2(n)$ we have

$$X_2(z) = \sum_{n=-\infty}^{+\infty} [0.4^n u(n)]\, z^{-n} = \sum_{n=0}^{+\infty} \left(0.4z^{-1}\right)^n$$

This geometric series converges if and only if

$$|0.4z^{-1}| < 1 \Rightarrow |z| > 0.4$$

Thus $\text{ROC}_2 : |z| > 0.4$.
We have

$$x(n) = x_1(n) + x_2(n)$$

then the z-Transform of $x(n)$ is

$$\begin{aligned} X(z) &= X_1(z) + X_2(z) \\ &= \sum_{n=0}^{+\infty} \left(0.5z^{-1}\right)^n + \sum_{n=0}^{+\infty} \left(0.4z^{-1}\right)^n \\ &= \sum_{n=0}^{+\infty} \left[\left(0.5z^{-1}\right)^n + \left(0.4z^{-1}\right)^n\right]. \end{aligned}$$

By linearity, we have

$$\begin{aligned} \text{ROC} &= \text{ROC}_1 \cap \text{ROC}_2 \\ &= \{z \ : \ |z| > 0.5\} \cap \{z \ : \ |z| > 0.4\} \\ &= \{z \ : \ |z| > 0.5\} \end{aligned}$$

4.3 Exercise 3

Question: Find the z-Transform and the ROC of the following discrete-time signal:

$$x(n) = 0.5^n u(n) + 0.9^n u(-n-1)$$

Answer: The discrete-time sequence $x(n)$ is a two-sided sequence composed of the linear combination of a causal (right-sided) sequence and an anti-causal (left-sided) sequence:

$$x(n) = x_1(n) + x_2(n),$$

where $x_1(n) = 0.5^n u(n)$ and $x_2(n) = 0.9^n u(-n-1)$.

4.3.0.1 Z-Transform of the Causal Component $x_1(n)$: For the right-sided sequence $x_1(n) = 0.5^n u(n)$:

$$X_1(z) = \sum_{n=-\infty}^{+\infty} \left[0.5^n u(n)\right] z^{-n} = \sum_{n=0}^{+\infty} \left(0.5z^{-1}\right)^n.$$

This geometric series converges if and only if the magnitude of the common ratio is strictly less than 1:

$$|0.5z^{-1}| < 1 \implies |z| > 0.5.$$

Thus, $\text{ROC}_1 = \{z \in \mathbb{C} : |z| > 0.5\}$. Under this condition, evaluating the infinite geometric sum $\sum_{n=0}^{\infty} q^n = \frac{1}{1-q}$ with $q = 0.5z^{-1}$:

$$X_1(z) = \frac{1}{1 - 0.5z^{-1}} = \frac{z}{z - 0.5}.$$

4.3.0.2 Z-Transform of the Anti-Causal Component $x_2(n)$: The unit step $u(-n-1)$ is non-zero only when $-n - 1 \geq 0 \iff n \leq -1$. Therefore:

$$X_2(z) = \sum_{n=-\infty}^{+\infty} \left[0.9^n u(-n-1)\right] z^{-n} = \sum_{n=-\infty}^{-1} 0.9^n z^{-n}.$$

Applying the change of summation index $m = -n$ (where m ranges from 1 to $+\infty$):

$$X_2(z) = \sum_{m=1}^{+\infty} 0.9^{-m} z^m = \sum_{m=1}^{+\infty} \left(\frac{z}{0.9}\right)^m.$$

This geometric series converges if and only if:

$$\left|\frac{z}{0.9}\right| < 1 \implies |z| < 0.9.$$

Thus, $\text{ROC}_2 = \{z \in \mathbb{C} : |z| < 0.9\}$. When converging, using $\sum_{m=1}^{\infty} w^m = \frac{w}{1-w}$ with $w = \frac{z}{0.9}$:

$$X_2(z) = \frac{\frac{z}{0.9}}{1 - \frac{z}{0.9}} = \frac{z}{0.9 - z} = -\frac{z}{z - 0.9} = -\frac{1}{1 - 0.9z^{-1}}.$$

4.3.0.3 Combined Z-Transform and Region of Convergence: Applying the linearity property of the Z-transform, $X(z) = X_1(z) + X_2(z)$:

$$\begin{aligned}
X(z) &= \frac{1}{1 - 0.5z^{-1}} - \frac{1}{1 - 0.9z^{-1}} \\
&= \frac{z}{z - 0.5} - \frac{z}{z - 0.9} \\
&= \frac{z(z - 0.9) - z(z - 0.5)}{(z - 0.5)(z - 0.9)} \\
&= \frac{z^2 - 0.9z - z^2 + 0.5z}{(z - 0.5)(z - 0.9)} \\
&= \frac{-0.4z}{(z - 0.5)(z - 0.9)} = \frac{-0.4z^{-1}}{(1 - 0.5z^{-1})(1 - 0.9z^{-1})}.
\end{aligned}$$

By linearity, the overall Region of Convergence is the intersection of the individual ROCs:

$$\begin{aligned}\text{ROC} &= \text{ROC}_1 \cap \text{ROC}_2 \\ &= \{z \in \mathbb{C} : |z| > 0.5\} \cap \{z \in \mathbb{C} : |z| < 0.9\} \\ &= \{z \in \mathbb{C} : 0.5 < |z| < 0.9\}.\end{aligned}$$

Analytical Observations:

- **Annular Geometry:** Because $x(n)$ is a two-sided sequence, its ROC is an open annular ring bounded on the interior by the pole at $z = 0.5$ and on the exterior by the pole at $z = 0.9$.
- **BIBO Stability and DTFT Convergence:** Crucially, the unit circle $|z| = 1$ is *not* included within the ROC since $|z| < 0.9 < 1$. Consequently, the sequence does not possess a convergent Discrete-Time Fourier Transform (DTFT), and an LTI system characterized by the impulse response $h(n) = x(n)$ is BIBO unstable.

4.4 Exercise 4

Question: Investigate the existing library or toolbox in Scilab to process audio data.
Answer: Scilab provides both built-in functions in its standard library and external modules via ATOMS (such as the *Sound System Toolbox*) to record, synthesize, process, and analyze audio signals. The core built-in functions include:

- `wavread` / `loadwave`: Reads RIFF-WAV audio files into memory, returning the sampled data matrix, the sampling frequency (F_s), and the bit resolution.
- `wavwrite` / `savewave`: Encodes and saves numeric signal vectors into standard `.wav` audio files.
- `playsnd`: Interfaces directly with the operating system audio output device to play numeric waveforms.
- `mapsound`: Analyzes audio signals in the time-frequency domain and visualizes sound spectrograms.

Demonstration Script: To demonstrate audio processing without external file dependencies, a standard 440 Hz tone (musical note A4) is synthesized, attenuated with an exponential decay envelope, and plotted in the time domain:

```
// 1. Synthesize audio signal: 440 Hz tone, Fs = 8000 Hz, duration 0.5 s
Fs = 8000;
t = 0 : 1/Fs : 0.5;
f0 = 440;
audio_orig = sin(2 * %pi * f0 * t);

// 2. Audio manipulation: Exponential amplitude attenuation
decay = exp(-6 * t);
audio_proc = audio_orig .* decay;

```

```
// 3. Export to WAV file (optional playback via playsnd)
wavwrite(audio_proc, Fs, "audio_tone.wav");

// 4. Time-domain visualization (first 100 samples)
figure();
subplot(2, 1, 1);
plot(t(1:100), audio_orig(1:100));
title("Original Audio Signal (440 Hz)");
xlabel("Time (s)"); ylabel("Amplitude");

subplot(2, 1, 2);
plot(t(1:100), audio_proc(1:100));
title("Attenuated Audio Signal");
xlabel("Time (s)"); ylabel("Amplitude");
```

Listing 1: Audio signal generation and amplitude attenuation in Scilab

Remarks: The discrete audio waveform is represented as a standard 1D array in Scilab. Pointwise vector operations (`.*`) enable straightforward time-domain processing such as volume adjustments, envelope shaping, echo effects, and filtering before audio playback or file export.

4.5 Exercise 5

Question: Investigate the existing library or toolbox in Scilab for image processing. Then, do an example or a demo to show a simple manipulation (e.g., Histogram display, histogram equalization, blur, and watermarking) on a digital image.
Answer: Scilab provides several dedicated open-source toolboxes available via its ATOMS package manager to process 2D/3D digital images and perform computer vision tasks:

1. **IPCV (Image Processing and Computer Vision Toolbox):** The most comprehensive and widely adopted vision package in Scilab. Maintained by Vincent Couvert and Jonathan Rouzaud-Cornabas, IPCV includes over 150 functions:
    - File I/O and display: `imread`, `imwrite`, `imshow`, `imsave`.
    - Color conversions: `rgb2gray`, `gray2ind`, `rgb2hsv`.
    - Intensity enhancements: `imhist` (histogram computation), `imhistequal` (histogram equalization), `imadjust` (dynamic range stretching).
    - Spatial filtering: `fspecial` (creating Gaussian, Sobel, Laplacian kernels), `imfilter` (2D spatial convolution and correlation).
    - Image arithmetic: `imlincomb` (linear combination/blending), `imadd`, `imsubtract`, `immultiply`.
    - Morphological operations: `imdilate`, `imerode`, `imopen`, `imclose`.
2. **SIVP (Scilab Image and Video Processing Toolbox):** A high-performance toolbox interfacing directly with OpenCV C++ libraries. It specializes in real-time camera video capture, video file decoding, optical flow estimation, and edge tracking.
3. **SciCV (Scilab Computer Vision Toolbox):** A modern bridge interfacing recent OpenCV releases (OpenCV 4.x) with Scilab data structures.

4.5.0.1 ATOMS Image Processing Package Installation via Bash: Scilab provides the ATOMS (AuTOmatic Modules Management for Scilab) package repository system. While modules can be installed interactively via the Scilab GUI or prompt, in automated test environments, continuous integration, or headless systems, toolboxes can be downloaded and installed non-interactively using a Bash script interfacing with `scilab-cli`:

```
#!/usr/bin/env bash
# ============================================================================
# Script: install_atoms_image.sh
# Description: Automated installation of IPCV toolbox via Scilab CLI
# ============================================================================

if ! command -v scilab-cli &> /dev/null; then
    echo "[ERROR] scilab-cli not found in system PATH."
    exit 1
fi

echo "[INFO] Updating Scilab ATOMS repository metadata..."
scilab-cli -nb -quit -e "atomsSystemUpdate();"

echo "[INFO] Installing IPCV (Image Processing and Computer Vision) toolbox..."
scilab-cli -nb -quit -e "atomsInstall('IPCV');"

echo "[INFO] Verifying IPCV installation and auto-loading..."
scilab-cli -nb -quit -e "if atomsIsLoaded('IPCV') then disp('IPCV is active'); else
    atomsLoad('IPCV'); disp('IPCV loaded successfully'); end"

echo "[SUCCESS] Image processing toolbox installed and ready for use!"
```

Listing 2: Bash script to install and load the IPCV toolbox via ATOMS

Within the Scilab interactive terminal, the equivalent commands are:

```
atomsSystemUpdate();       // Update module catalog
atomsInstall("IPCV");      // Download and install IPCV
atomsLoad("IPCV");         // Load IPCV into active session
atomsIsLoaded("IPCV")      // Returns %T if successfully loaded
```

Listing 3: Scilab console commands for ATOMS toolbox management

4.5.0.2 Demonstration: Digital Image Watermarking via Alpha Blending: Digital watermarking embeds copyright logos or ownership signatures into a host image to assert intellectual property rights. Alpha blending combines a host image $H(x, y)$ and a watermark logo $W(x, y)$ resized to matching spatial dimensions $[h, w]$:

$$I_{\text{watermarked}}(x, y) = \alpha \cdot H(x, y) + (1 - \alpha) \cdot W(x, y), \tag{9}$$

where $\alpha \in [0, 1]$ controls the blending ratio. Choosing $\alpha = 0.8$ preserves 80% of the host image luminance and color saturation while embedding 20% of the watermark logo, making the mark distinctly identifiable without obscuring essential features of the host.

4.5.0.3 Scilab Script Implementation: The watermarking demonstration is implemented using Scilab and the IPCV toolbox:

```
atomsLoad("IPCV");

// 1. Load host image and logo/watermark from IPCV library
host = imread(fullpath(getIPCVpath() + "/images/peppers.png"));
logo = imread(fullpath(getIPCVpath() + "/images/logo.png"));

// 2. Resize logo to match host spatial dimensions [h, w]
[h, w, c] = size(host);
logo_resized = imresize(logo, [h, w]);

// 3. Alpha blend: Watermarked = 0.8 * Host + 0.2 * Watermark
watermarked = imlincomb(0.8, host, 0.2, logo_resized);

// 4. Display the original, watermark, and blended result
figure("Figure_Name", "Watermarking / Alpha Blending");
subplot(1, 3, 1); imshow(host);         title("Host Image");
subplot(1, 3, 2); imshow(logo_resized); title("Watermark");
subplot(1, 3, 3); imshow(watermarked);  title("Watermarked Image (80/20 Blend)");
```

Listing 4: Digital image watermarking and alpha blending in Scilab (IPCV)

Experimental Results: The three-panel output generated from the Scilab script is shown in Figure 1:



Figure 1: Digital image watermarking demonstration: (left) host image, (center) resized watermark logo, (right) watermarked image with an 80/20 alpha blend.

Remarks: In Scilab, images are represented as multi-dimensional numerical tensors of type `uint8` with values in the range $[0, 255]$. The `imlincomb` function performs saturation arithmetic, automatically clamping intermediate sum values to 255 and preventing numerical wrap-around overflow artifacts.

4.6 Exercise 6

Question: Let there are two discrete signals $x(n), h(n), n = 0, \pm 1, \pm 2, \ldots$, defined over the entire range of n. Then the linear convolution of discrete signals is $y(n)$ of the form

$$y(n) = \sum_{k=-\infty}^{\infty} h(k)x(n-k).$$

1. Derive the signal $y(n)$ such that $y(n)$ converges in the following cases.
   - Both $h(n)$ and $x(n)$ are causal.
   - The system has a finite impulse response such that $h(n) = 0, n < 0 \vee n > M$.
2. Suppose the second case is hold, compute the convolution $y(n)$ of $x(n) = [1 \uparrow, 2, -3, 2, 1]$ and $h(n) = [1 \uparrow, 0, -1]$ using the "folding and shifting" method. Implement your answer with Scilab script.
3. Suppose the second case is hold, compute the convolution $y(n)$ of $x(n) = [1 \uparrow, 2, -3, 2, 1]$ and $h(n) = [1 \uparrow, 0, -1]$ using the matrix method. Implement your answer with Scilab script.
4. Drawing $x(n)$, $y(n)$ and energy of discrete signals in the same figure.

Answer:

1. Case 1: Both $h(n)$ and $x(n)$ are causal.

   For $h(n)$ and $x(n)$ to be causal, their value must be zero for $n < 0$. Applying to $y(n)$, $h(n)$ causality restricts the lower bound for k to 0, while $x(n)$'s causality restricts the upper bound for k to n. Thus

   $$y(n) = \sum_{k=0}^{n} h(k)x(n-k)$$

   Case 2: The system has a finite impulse response such that $h(n) = 0, n < 0 \vee n > M$.

   Because $h(n)$ is only nonzero for $0 \leq n \leq M$, the range of k is restricted to $[0, M]$. Thus

   $$y(n) = \sum_{k=0}^{M} h(k)x(n-k)$$

   Because there can only be at most $M + 1$ terms in the summation above for each n, $y(n)$ is finite provided that $x(n-k)$ is bounded.

2. We have the impulse response $h(n) = [1 \uparrow, 0, -1]$ with $N_h = 3$, $n_{min}(h) = 0$, $n_{max}(h) = 2$ and the input signal $h(n) = [1 \uparrow, 2, -3, 2, 1]$ with $N_x = 5$, $n_{min}(x) = 0$, $n_{max}(x) = 4$. Before proceeding with the convolution, we may calculate the expected length of the response $y(n)$ and its time range, which are respectively, $L_y = N_x + N_h - 1 = 5 + 3 = 7$, $n_{min}(y) = n_{min}(x) + n_{min}(h) = 0$, and $n_{max}(y) = n_{max}(x) + n_{max}(h) = 6$.

We have

$$y(n) = \sum_{k=0}^{2} h(k)x(n-k) = h(0)x(n)+h(1)x(n-1)+h(2)x(n-2) = x(n)-x(n-2) \quad (10)$$

From (10) we can caluclate $y(n)$ at each $n \in [0, 6]$

$$\begin{aligned}
y(0) &= x(0) - x(-2) = 1 \\
y(1) &= x(1) - x(-1) = 2 \\
y(2) &= x(2) - x(0) = -3 - 1 = -4 \\
y(3) &= x(3) - x(1) = 2 - 2 = 0 \\
y(4) &= x(4) - x(2) = 1 + 3 = 4 \\
y(5) &= x(5) - x(3) = -2 \\
y(6) &= x(6) - x(4) = -1
\end{aligned}$$

Thus, $y(n) = [1\uparrow, 2, -4, 0, 4, -2, -1]$.

3. The discrete convolution $y = x * h$ can be represented in matrix form as

$$\mathbf{y} = \mathbf{H}\mathbf{x} \quad (11)$$

where the input $\mathbf{x}$ is an $N_x \times 1$ column vector, the ouput $\mathbf{y}$ is an $L_y \times 1$ column vector, and the convolution matrix $\mathbf{H}$ has dimensions $L_y \times N_x$.

The convolution matrix $\mathbf{H}$ is built from the impulse response $h = [1, 0, -1]^T$, where the first column starts with $h(0)$, $h(1)$, $h(2)$, followed by zeros down to length $L_y = 7$. Each subsequent column is the previous column shifted down by one position with zeros padded at the top.

For the above convolution, the matrices are

$$\mathbf{y}_{7\times 1} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 \end{bmatrix} \times \begin{bmatrix} 1 \\ 2 \\ -3 \\ 2 \\ 1 \end{bmatrix} \quad (12)$$

Each row in $\mathbf{y}$ from topdown corresponds to the value of $y(n)$ for $n \in [0, 6]$. From (12) we can calculate

$$\begin{aligned}
y(0) &= 1 \times 1 + 0 + 0 + 0 + 0 = 1 \\
y(1) &= 0 + 1 \times 2 + 0 + 0 + 0 = 2 \\
y(2) &= (-1) \times 1 + 0 + 1 \times (-3) + 0 + 0 = -3 - 1 = -4 \\
y(3) &= 0 + (-1) \times 2 + 0 + 1 \times 2 + 0 = 2 - 2 = 0 \\
y(4) &= 0 + 0 + (-1) \times (-3) + 0 + 1 \times 1 = 1 + 3 = 4 \\
y(5) &= 0 + 0 + 0 + (-1) \times 2 + 0 = -2 \\
y(6) &= 0 + 0 + 0 + 0 + (-1) \times 1 = -1
\end{aligned}$$

Thus, $y(n) = [1\uparrow, 2, -4, 0, 4, -2, -1]$.

4. In this part we will write the script to verify the results in part 2 and 3, plot $x(n)$, $y(n)$ and their energies.

   Before we begin, let's define the signals as row vectors:

```
x = [1, 2, -3, 2, 1];
h = [1, 0, -1];

Lx = length(x);
Lh = length(h);
Ly = Lx + Lh - 1;
```

   The script to perform $y = x * h$ by folding and shifting is given below:

```
// Initialize output vector y
y_fold = zeros(1, Ly);

// Zero-pad x so index lookups don't go out of bounds
// We pad with (Lh - 1) zeros on both sides
x_padded = [zeros(1, Lh - 1), x, zeros(1, Lh - 1)];

// h reversed (folded): h(-k)
h_folded = h($:-1:1);

// Slide and multiply
for n = 1:Ly
    // Extract the window of x that aligns with the folded h
    segment = x_padded(n : n + Lh - 1);
    y_fold(n) = sum(segment .* h_folded);
end

disp("y(n) via Folding and Shifting:");
disp(y_fold);
```

   Which returned

```
"y(n) via Folding and Shifting:"
1.   2.  -4.   0.   4.  -2.  -1.
```

   The script to perform $y = x * h$ by using matrices is

```
// Column 1 of H: h padded with zeros to length Ly
col1 = [h(:); zeros(Ly - Lh, 1)];

// Row 1 of H: h(1) followed by (Lx - 1) zeros
row1 = [h(1), zeros(1, Lx - 1)];

// Build the Toeplitz convolution matrix H (size 7 x 5)
H = toeplitz(col1, row1);

// Convert x to a column vector and multiply
x_col = x(:);
y_matrix = H * x_col;

disp("Convolution Matrix H:");
```

```
disp(H);

disp("y(n) via Matrix Multiplication:");
disp(y_matrix');
```

Whose result is

```
"Convolution Matrix H:"
 1.    0.    0.    0.    0.
 0.    1.    0.    0.    0.
-1.    0.    1.    0.    0.
 0.   -1.    0.    1.    0.
 0.    0.   -1.    0.    1.
 0.    0.    0.   -1.    0.
 0.    0.    0.    0.   -1.
"y(n) via Matrix Multiplication:"
 1.    2.   -4.    0.    4.   -2.   -1.
```

The convolution results agree with the calculations in part 2 and 3.

Finally the plots of $x(n)$, $y(n)$ and their energies are illustrated in the figure below:

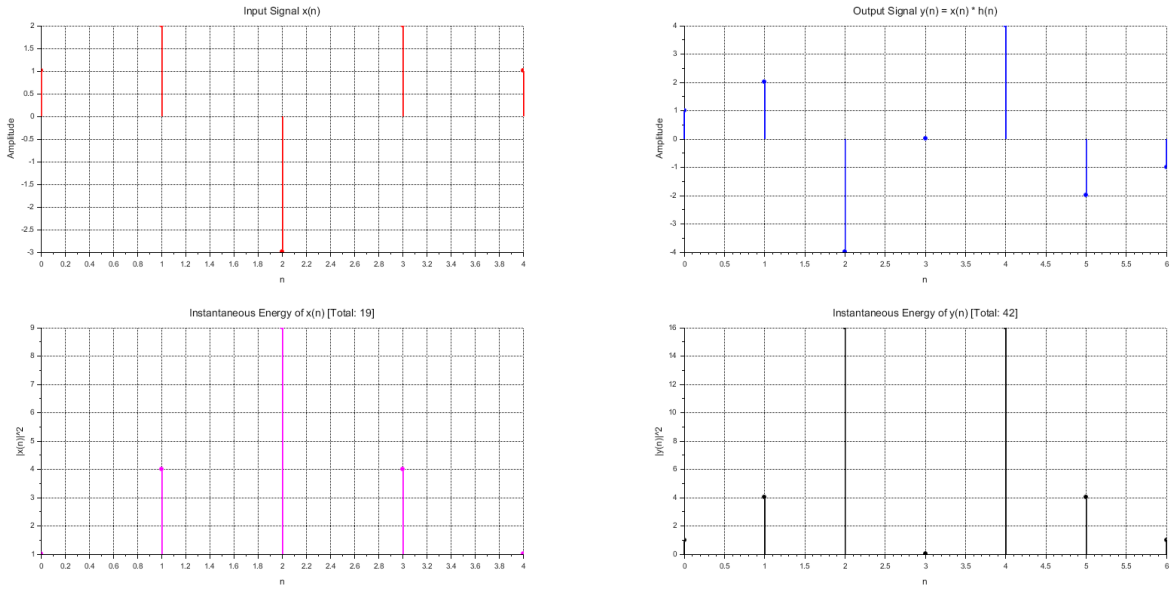


Figure 2: $x(n)$ and $y(n)$ with their energies respectively.

4.6.0.1 ATOMS Signal Processing Setup and Headless Batch Execution via Bash:

To run the discrete-time convolution analysis headlessly in server or automated grading environments, a Bash script is utilized to ensure required ATOMS signal modules are installed and execute the Scilab script in batch mode:

```
#!/usr/bin/env bash
#
    ================================================================================

# Script: setup_and_run_ex6.sh
```

```
# Description: Automated ATOMS setup and batch execution of Exercise 6
#
    ================================================================================


# Verify Scilab command-line executable exists
if ! command -v scilab-cli &> /dev/null; then
    echo "[ERROR] scilab-cli is not installed or not in system PATH."
    exit 1
fi

# 1. Update ATOMS repository index
echo "[INFO] Updating ATOMS package repositories..."
scilab-cli -nb -quit -e "atomsSystemUpdate();"

# 2. Install optional signal processing & numerical extensions
echo "[INFO] Installing ATOMS signal processing modules..."
scilab-cli -nb -quit -e "atomsInstall('signal');"

# 3. Query and verify all installed ATOMS toolboxes
echo "[INFO] Verifying installed ATOMS modules:"
scilab-cli -nb -quit -e "atomsDispInstalled();"

# 4. Execute the linear convolution script in headless batch mode
echo "[INFO] Executing linear convolution simulation (lap4_ex6.sci)..."
scilab-cli -nb -f src/lap4_ex6.sci

echo "[SUCCESS] Batch convolution execution completed successfully!"
```

Listing 5: Bash script for ATOMS package setup and headless execution in Exercise 6

4.7 Exercise 7

Question: For a periodic signal $x(n)$ of length N and impulse response $h(n)$ of length M, the circular convolution between $x(n)$ and $h(n)$ is defined as

$$y(n) = \sum_{k=0}^{M-1} h(k)x((n-k) \bmod N).$$

Explain why $y(n)$ can be derived that way.

1. Compute the convolution $y(n)$ of $x(n) = [1 \uparrow, 2, -3, 2, 1]$ and $h(n) = [1 \uparrow, 0, -1, -1, 1]$ using the "folding and shifting" method. Implement your answer with Scilab script.

2. Compute the convolution $y(n)$ of $x(n) = [1 \uparrow, 2, -3, 2, 1]$ and $h(n) = [1 \uparrow, 0, -1, -1, 1]$ using the matrix method. Implement your answer with Scilab script.

3. Drawing $x(n)$, $y(n)$ and energy of discrete signals in the same figure.

Answer:

4.7.0.1 Theoretical Derivation of Circular Convolution: Consider a discrete-time signal $\tilde{x}(n)$ that is strictly periodic with fundamental period N, such that $\tilde{x}(n) = \tilde{x}(n + rN)$ for all integers r. When this periodic signal is fed into a linear time-invariant (LTI) system with a finite impulse response $h(n)$ of length M, the output sequence $\tilde{y}(n)$ is obtained via the linear convolution sum:

$$\tilde{y}(n) = \sum_{k=-\infty}^{+\infty} h(k)\tilde{x}(n-k) = \sum_{k=0}^{M-1} h(k)\tilde{x}(n-k).$$

Because $\tilde{x}(n)$ is N-periodic, the argument $n-k$ can be replaced by its equivalent modulo residue $m = (n-k) \bmod N \in [0, N-1]$:

$$\tilde{x}(n-k) = x((n-k) \bmod N),$$

where $x(n)$ is the primary period sequence defined on $n \in [0, N-1]$. Restricting our attention to the fundamental period of the output $n \in [0, N-1]$, the convolution sum becomes:

$$y(n) = \sum_{k=0}^{M-1} h(k)x((n-k) \bmod N), \qquad 0 \le n \le N-1. \tag{13}$$

Furthermore, in the transform domain, taking the N-point Discrete Fourier Transform (DFT) of both sequences yields:

$$Y(k) = X(k) \cdot H(k), \qquad k = 0, 1, \ldots, N-1.$$

By the circular convolution theorem of the DFT, pointwise multiplication of DFT spectra corresponds in the discrete-time domain directly to circular convolution, which aliases the linear convolution sequence $y_{\text{linear}}(n)$:

$$y(n) = \sum_{r=-\infty}^{+\infty} y_{\text{linear}}(n + rN), \qquad 0 \le n \le N-1. \tag{14}$$

Hence, equation (13) is the exact mathematical formulation of circular convolution.

1. **Folding and Shifting Method:**
   We are given the two discrete sequences of length $N = M = 5$:

   $$x(n) = [1\uparrow, 2, -3, 2, 1] \implies x(0) = 1,\ x(1) = 2,\ x(2) = -3,\ x(3) = 2,\ x(4) = 1,$$
   $$h(n) = [1\uparrow, 0, -1, -1, 1] \implies h(0) = 1,\ h(1) = 0,\ h(2) = -1,\ h(3) = -1,\ h(4) = 1.$$

   Expanding the summation for $N = 5$:

   $$\begin{aligned} y(n) &= \sum_{k=0}^{4} h(k)x((n-k) \bmod 5) \\ &= 1 \cdot x(n) + 0 \cdot x((n-1) \bmod 5) - 1 \cdot x((n-2) \bmod 5) \\ &\quad - 1 \cdot x((n-3) \bmod 5) + 1 \cdot x((n-4) \bmod 5) \\ &= x(n) - x((n-2) \bmod 5) - x((n-3) \bmod 5) + x((n-4) \bmod 5). \end{aligned}$$

Evaluating $y(n)$ step-by-step for each $n \in \{0, 1, 2, 3, 4\}$:

$$\begin{aligned}
y(0) &= x(0) - x(3) - x(2) + x(1) = 1 - 2 - (-3) + 2 = 1 - 2 + 3 + 2 = 4, \\
y(1) &= x(1) - x(4) - x(3) + x(2) = 2 - 1 - 2 + (-3) = 2 - 1 - 2 - 3 = -4, \\
y(2) &= x(2) - x(0) - x(4) + x(3) = -3 - 1 - 1 + 2 = -3, \\
y(3) &= x(3) - x(1) - x(0) + x(4) = 2 - 2 - 1 + 1 = 0, \\
y(4) &= x(4) - x(2) - x(1) + x(0) = 1 - (-3) - 2 + 1 = 1 + 3 - 2 + 1 = 3.
\end{aligned}$$

Thus, the circular convolution sequence is:

$$y(n) = [4 \uparrow, -4, -3, 0, 3]. \tag{15}$$

2. **Matrix Method:**
Circular convolution can be formulated as a matrix-vector product:

$$\mathbf{y} = \mathbf{X}_c \mathbf{h}, \tag{16}$$

where $\mathbf{h} = [1, 0, -1, -1, 1]^T$ and $\mathbf{X}_c$ is a 5×5 circulant matrix formed by circularly shifting $x(n)$:

$$\mathbf{X}_c = \begin{bmatrix} x(0) & x(4) & x(3) & x(2) & x(1) \\ x(1) & x(0) & x(4) & x(3) & x(2) \\ x(2) & x(1) & x(0) & x(4) & x(3) \\ x(3) & x(2) & x(1) & x(0) & x(4) \\ x(4) & x(3) & x(2) & x(1) & x(0) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 2 & -3 & 2 \\ 2 & 1 & 1 & 2 & -3 \\ -3 & 2 & 1 & 1 & 2 \\ 2 & -3 & 2 & 1 & 1 \\ 1 & 2 & -3 & 2 & 1 \end{bmatrix}. \tag{17}$$

Performing the matrix multiplication:

$$\mathbf{y} = \begin{bmatrix} y(0) \\ y(1) \\ y(2) \\ y(3) \\ y(4) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 2 & -3 & 2 \\ 2 & 1 & 1 & 2 & -3 \\ -3 & 2 & 1 & 1 & 2 \\ 2 & -3 & 2 & 1 & 1 \\ 1 & 2 & -3 & 2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ -1 \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ -4 \\ -3 \\ 0 \\ 3 \end{bmatrix}. \tag{18}$$

The matrix calculation yields $y(n) = [4 \uparrow, -4, -3, 0, 3]$, which exactly matches the folding and shifting result.

3. **Signal Energy and Scilab Implementation:**
The energy of a discrete-time signal $s(n)$ of length N is defined as $E_s = \sum_{n=0}^{N-1} |s(n)|^2$:

$$\begin{aligned}
E_x &= \sum_{n=0}^{4} |x(n)|^2 = 1^2 + 2^2 + (-3)^2 + 2^2 + 1^2 = 1 + 4 + 9 + 4 + 1 = 19, \\
E_y &= \sum_{n=0}^{4} |y(n)|^2 = 4^2 + (-4)^2 + (-3)^2 + 0^2 + 3^2 = 16 + 16 + 9 + 0 + 9 = 50.
\end{aligned}$$

The complete Scilab script executing the folding and shifting method, the matrix formulation, energy evaluations, and stem-plot generation is given below:

```
clear; clc; clf;

// 1. Data definition
x = [1, 2, -3, 2, 1];      // x(0)=1, length N=5
h = [1, 0, -1, -1, 1];     // h(0)=1, length M=5
N = length(x);
M = length(h);
n = 0 : N - 1;

// 2. Folding and shifting method
y_fs = zeros(1, N);
for i = 0 : N - 1
    idx = pmodulo(i - (0 : M - 1), N) + 1; // 1-based indexing
    x_fs = x(idx);
    y_fs(i + 1) = sum(h .* x_fs);
end
disp("y via Folding and Shifting:");
disp(y_fs);

// 3. Matrix method (Circulant Matrix)
Xc = zeros(N, M);
for i = 0 : N - 1
    for k = 0 : M - 1
        Xc(i + 1, k + 1) = x(pmodulo(i - k, N) + 1);
    end
end
y_mat = (Xc * h')';
disp("y via Matrix Method:");
disp(y_mat);

// 4. Energy calculations
ex = abs(x).^2;  Ex = sum(ex);
ey = abs(y_mat).^2;  Ey = sum(ey);
mprintf("Energy of x = %g, Energy of y = %g\n", Ex, Ey);

// 5. Plotting discrete signals and energy
scf(0); clf;
subplot(2, 1, 1);
plot2d3([n' - 0.08, n' + 0.08], [x', y_mat'], style=[2, 5], rect=[-0.5, -5, N -
    0.5, 5]);
xgrid(); xlabel("n"); ylabel("amplitude");
title("x(n) and y(n) = circular convolution of x and h");
legend(["x(n)"; "y(n)"], 1);

subplot(2, 1, 2);
plot2d3([n' - 0.08, n' + 0.08], [ex', ey'], style=[2, 5], rect=[-0.5, 0, N -
   0.5, 20]);
xgrid(); xlabel("n"); ylabel("|.|^2");
title(msprintf("Energy per sample: Ex = %g, Ey = %g", Ex, Ey));
```

```
legend(["|x(n)|^2"; "|y(n)|^2"], 1);
```

Listing 6: Circular convolution, energy computation, and visualization in Scilab

The execution log verified identity between both methods and analytical values:

```
"y via Folding and Shifting:"
 4.  -4.  -3.   0.   3.
"y via Matrix Method:"
 4.  -4.  -3.   0.   3.
Energy of x = 19, Energy of y = 50
```

The resulting graphical stem plots illustrating the signal waveforms and energy distributions are displayed in Figure 3:

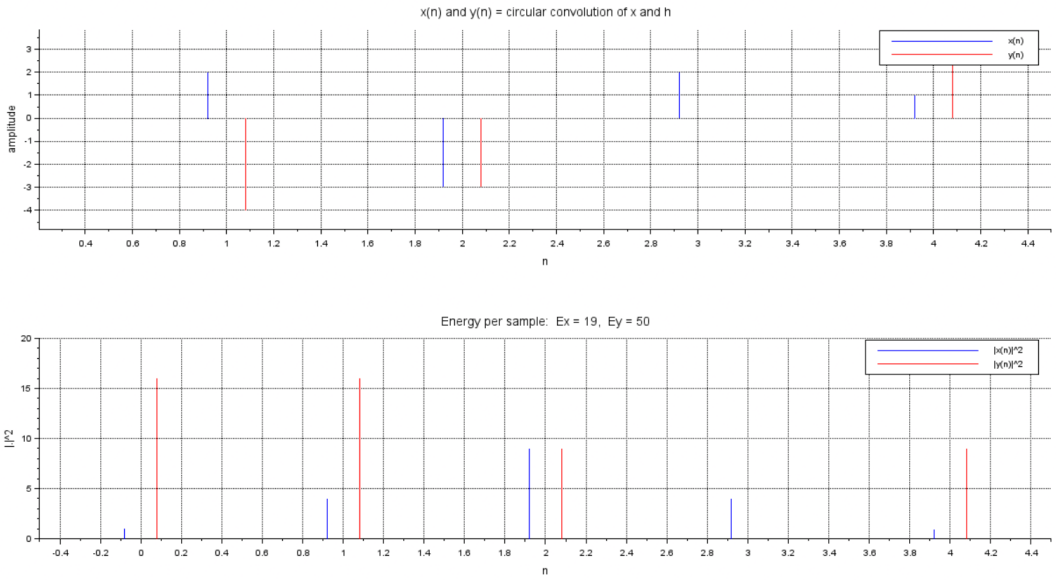


Figure 3: Circular convolution $y(n) = x(n) \circledast h(n)$ and discrete sample energies $E_x = 19$ and $E_y = 50$.

5 Discussion

The theoretical derivations and Scilab computational experiments conducted throughout Laboratory 4 provide several key insights:

1. **Z-Transform and ROC Characteristics:**
   - *Finite-Duration Sequences (Exercise 1):* As demonstrated analytically, finite-length sequences converge over the entire complex plane, with the only possible exceptions at $z = 0$ (if $x(n)$ contains terms for $n > 0$) and $z = \infty$ (if $x(n)$ contains terms for $n < 0$). Because the unit circle $|z| = 1$ is guaranteed to lie within the ROC for any finite-length bounded signal, its DTFT always exists.
   - *Exponential Sequences and Stability (Exercises 2 & 3):* For infinite-duration sequences, the ROC geometry fundamentally dictates causality and stability. Causal

sequences converge outside an exterior circle ($|z| > |a|$), while anti-causal sequences converge inside an interior disk ($|z| < |b|$). When summing causal and anti-causal components (Exercise 3), an annular region $0.5 < |z| < 0.9$ is formed. Since this ring does not enclose the unit circle $|z| = 1$, the resulting sequence is BIBO unstable and lacks a convergent Fourier transform.

2. **Linear versus Circular Convolution (Exercises 6 & 7):**
   - *Length and Boundary Effects:* For sequences of lengths N_x and N_h, linear convolution produces an extended sequence of length $L = N_x + N_h - 1 = 5 + 3 - 1 = 7$ (Exercise 6). Conversely, circular convolution operates over a fixed modulo base $N = 5$ (Exercise 7), resulting in an output of strictly length $N = 5$.
   - *Time-Domain Aliasing:* Circular convolution inherently wraps the tail of the linear convolution back into the primary period: $y_{\text{circ}}(n) = \sum_r y_{\text{linear}}(n + rN)$. To perform linear convolution via circular convolution (such as when utilizing FFT-based fast convolution), both sequences must be zero-padded to at least length $L \geq N_x + N_h - 1$ prior to computing the circular convolution.
   - *Matrix Representations:* Linear convolution corresponds to multiplication by a banded lower-triangular Toeplitz matrix $\mathbf{H}$, whereas circular convolution corresponds to multiplication by a circulant matrix $\mathbf{X}_c$, whose eigenvectors are the discrete Fourier basis vectors.
3. **Scilab Multimedia Processing Capabilities (Exercises 4 & 5):** Scilab proves to be a versatile platform for signal and multimedia engineering. Audio data is naturally handled as 1D arrays with built-in functions (`wavread`, `wavwrite`, `playsnd`), allowing rapid spatial envelope synthesis and filtering. For 2D image processing, the IPCV toolbox bridges high-level matrix manipulation with computer vision workflows (alpha blending, watermarking, spatial filtering), executing pixel-wise saturation arithmetic and spatial transformations cleanly.

6 References

- J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed., Pearson, 2007 – Chapter 2 Problems.
- Digital Signal Processing Lecture Notes - Ho Chi Minh City University of Technology.

7 Source Code

The source code, scripts, and LaTeX files for this laboratory report can be accessed at our GitHub repository:
https://github.com/aulequangtri/signal_scilab